

Student 23 **MALLI CHANDANA** 192472250RD

5 / 5 submitted

Q1. ATM Withdrawal – Exception Handling Debugging

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc =new Scanner(System.in);
        try{
            int balance=sc.nextInt();
            int amount=sc.nextInt();
            if(amount==0){
                throw new ArithmeticException("withdrawal amount cannot be zero");
            }
            if(amount <0){
                System.out.println("Inavlid withdrawal amount");
            }
            else if(amount>balance){
                System.out.println("Insufficient balnce");
            }
            else{
                int remainingBalance=balance-amount;
                System.out.println("withdrawal succesful");
                System.out.println("Remaining balance = "+remainingBalance);
            }
        }
        catch(InputMismatchException e){
            System.out.println("Invalid input");
        }
        catch(ArithmeticException e){
            System.out.println(e.getMessage());
        }
        catch(Exception e){
            System.out.println("Transaction failed");
        }
        finally{
            System.out.println("Transaction completed");
        }
    }
}
```

INPUT

10000
3000

OUTPUT

withdrawal succesful
Remaining balance = 7000
Transaction completed

Q2. Hospital Registration – NullPointerException Debugging

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            String inputName=sc.nextLine();
            String name;
            if(inputName.equals("NULL")){
                name = null;
            }else{
                name=inputName;
            }
            int age=sc.nextInt();
            if(name==null){
                System.out.println("patient name unavailable");
            }else{
                System.out.println("patient name: "+name.toUpperCase());
            }
            if(age>=18){
                System.out.println("Adult patient");
            }else{
                System.out.println("minor patient");
            }
        }catch(InputMismatchException e){
            System.out.println("Invalid age input");
        }
        catch(Exception e){
            System.out.println("Registration failed");
        }
        finally{
            System.out.println("Registration completed");
        }
    }
}
```

INPUT

Arun
25

OUTPUT

patient name: ARUN
Adult patient
Registration completed

Q3. E-Commerce Product Search – Array Exception Debugging

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String[] products = {"Laptop","Mobile","Tablet","Headphones","Keyboard"};
        try{
            int productNumber=sc.nextInt();
            if(productNumber<1||productNumber>5){
                System.out.println("Invalid product number");
            }else{
                System.out.println("selected product:"+products[productNumber-1]);
            }
        }
        catch(InputMismatchException e){
            System.out.println("Invalid input");
        }
        catch(Exception e){
            System.out.println("search failed");
        }
        finally{
            System.out.println("search completed");
        }
    }
}
```

INPUT

5

OUTPUT

selected product:Keyboard
search completed

Q4. Railway Reservation – Synchronization Debugging

CODE

```
import java.util.*;

public class Main{
    public static void main(String[] args){
        Reservation reservation=new Reservation();
        Customer c1=new Customer(reservation,"Customer 1");
        Customer c2=new Customer(reservation,"Customer 2");
        c1.start();
        c2.start();
    }
}

class Reservation{
    int seats=1;
    synchronized void bookSeat(String customer){
        if(seats>0){
            System.out.println(customer+"is booking a seat");
            try{
                Thread.sleep(100);
            }catch(InterruptedException e){
                System.out.println("Thread interrupted");
            }
            seats--;
            System.out.println(customer+"booking succesful");
        }else{
            System.out.println(customer+" - No seat available");
        }
    }
}

class Customer extends Thread{
    Reservation reservation;
    String customerName;
    Customer(Reservation reservation,String customerName){
        this.reservation=reservation;
        this.customerName=customerName;
    }
    public void run(){
        reservation.bookSeat(customerName);
    }
}
```

OUTPUT

```
Customer 1is booking a seat
Customer 1booking succesful
Customer 2 - No seat available
```

Q5. Bank Account – Race Condition Debugging

CODE

```
public class Main{
    public static void main(String[] args)throws InterruptedException{
        BankAccount account=new BankAccount();
        ATM atm1=new ATM(account,7000);
        ATM atm2=new ATM(account,6000);
        atm1.setName("ATM-1");
        atm2.setName("ATM-2");
        atm1.start();
        atm2.start();
        atm1.join();
        atm2.join();
        System.out.println("final balance ="+account.balance);
    }
}

class BankAccount{
    int balance = 10000;
    synchronized void withdraw(int amount){
        if(balance>=amount){
            System.out.println(Thread.currentThread().getName()+"withdraw"+amount);
            try{
                Thread.sleep(100);
            }catch(InterruptedException e){
                System.out.println("Interrupted");
            }
            balance=balance-amount;
            System.out.println(Thread.currentThread().getName()+"withdrawal succesful");
        }else{
            System.out.println(Thread.currentThread().getName()+"Insufficienr Balance");
        }
    }
}

class ATM extends Thread{
    BankAccount account;
    int amount;
    ATM(BankAccount account,int amount){
        this.account=account;
        this.amount=amount;
    }
    public void run(){
        account.withdraw(amount);
    }
}
```

OUTPUT

```
ATM-1withdraw7000
ATM-1withdrawal succesful
ATM-2Insufficienr Balance
final balance =3000
```